Today's AgentCore Learning Notes

Cognito, JWT, Runtime, Gateway, Lambda Tools and MCP

Simple revision handbook for a personal agent project

Login -> **Run Agent** -> **Choose Tool** -> **Secure Call** -> **Execute Business Logic** -> **Return Answer**

Main lesson

The customer logs in through Cognito. The frontend receives a JWT. AgentCore Runtime runs the agent. The Foundation Model chooses a tool. MCP carries remote tool calls. AgentCore Gateway authenticates, authorizes and routes. Lambda performs the business operation.

1. Complete Picture

The project contains multiple layers. Each layer has one clear responsibility.
Understanding the separation between these layers makes agent architecture much easier.

Customer -> **Frontend** -> **Cognito** -> **JWT** -> **Runtime** -> **MCP Client** -> **Gateway** -> **Lambda/API**

- Cognito manages user login and issues tokens.
- JWT proves that the user completed authentication.
- Runtime hosts and runs the complete coded agent.
- The Foundation Model understands the request and chooses tools.
- Local tools execute directly inside Runtime.
- MCP standardizes discovery and invocation of remote tools.
- Gateway exposes, secures and routes remote tools.
- Lambda contains the actual backend business logic.

> **Memory trick**
>
> Runtime thinks and coordinates. MCP communicates. Gateway checks and routes. Lambda performs the work.

2. Cognito and JWT

What	Cognito manages user accounts, verifies login credentials and issues tokens. JWT is the temporary signed token used after login.
Why	The application should not send or store the customer password for every request. A temporary token reduces password exposure and gives protected services a standard proof of authentication.
How	The customer enters credentials once. Cognito verifies them. The frontend receives a JWT and automatically attaches it to protected requests.

2.1 Does Cognito Create Usernames and Passwords?

- Self-registration: the customer chooses an email and password, verifies an OTP, and Cognito creates the account.
- Administrator creation: an administrator creates an account and assigns a temporary or permanent password.
- Cognito manages the user directory and password verification; application code should not retrieve customers' plain passwords.

2.2 What the Customer Sees vs What the App Does

Customer sees	Application does internally
Email and password	Sends login request to Cognito
Sign In button	Receives JWT after successful login
Normal application pages/chat	Automatically sends token with protected requests
Login again when required	Refreshes token or requests login after expiration

2.3 Token Reuse

A new token is not normally created for every activity. The same valid token or session is reused for many protected requests until it expires. Public actions may not require authentication; personal or sensitive actions do.

Login once -> Token issued -> Many protected requests -> Token expires -> Refresh or login again

Correct sentence

The customer uses credentials to log in. The frontend application uses the JWT after login.

3. AgentCore Runtime

What	AgentCore Runtime is the managed environment where the coded agent runs.
Why	The full agent needs secure compute, request handling, sessions, model access, memory integration, tools and response streaming.
How	Runtime receives a request, runs the Strands Agent, calls the model, executes selected tools and returns the final response.

3.1 Runtime Contains

- main.py and application dependencies
- Strands Agent
- Foundation Model connection
- System prompt
- Local Python tools
- MCP clients
- Memory session manager
- Session and response logic

3.2 How Runtime Knows Which Tool to Call

Runtime does not independently choose the tool. Runtime registers local tools and discovers MCP tools, then provides their names, descriptions and schemas to the Foundation Model. The model compares them with the user request and selects the best match.

User request -> Runtime provides context + tool descriptions -> Model selects tool -> Runtime executes request -> Model writes final answer

Responsibility split

Runtime provides and executes tools. The Foundation Model decides which tool is relevant. Clear tool descriptions make selection more accurate.

4. Local Tools vs Remote Tools

4.1 Local Tools

What	A local tool is a Python function packaged with the agent, such as get_product_info or get_return_policy.
Why	Local functions are fast and simple for small, agent-specific, low-risk logic.
How	The model selects the tool and Runtime executes the Python function directly. Gateway and MCP are not required for this direct local call.

Model chooses local tool -> Runtime calls Python function -> Function returns result -> Model answers

4.2 Remote Tools

A remote tool is implemented outside Runtime, such as a Lambda function, API or an existing MCP server. Runtime reaches it through an MCP client, often through AgentCore Gateway when centralized security and governance are required.

Model chooses remote tool -> MCP Client -> Gateway or MCP Server -> Backend operation -> Result returns

4.3 When to Use Which

- Use local tools for simple calculations, small static data and agent-specific helpers.
- Use remote tools for live databases, shared enterprise services, high-risk actions and capabilities maintained by other teams.
- A local tool can later be moved behind an API or Lambda and exposed through Gateway when the project grows.

5. Lambda Tools

What	A Lambda tool is a Lambda function containing backend business logic, such as warranty lookup or refund processing.
Why	The model cannot directly access company databases or safely perform business actions. A backend service must perform the actual operation.
How	Gateway invokes the Lambda with structured arguments after authentication, policy and guardrail checks pass.

5.1 Lambda, ARN and Tool Schema

- Lambda: the worker that performs the actual operation.
- Lambda ARN: the unique AWS address of that function.
- Tool schema: the instruction card describing the tool name, purpose, inputs and data types.
- Gateway target: the registration connecting the tool schema and Lambda ARN to a Gateway.

5.2 Why Use Lambda

- Reuse existing company business logic instead of copying it into agent code.
- Allow many agents to use the same capability.
- Let backend teams update the implementation independently.
- Access live databases and AWS services.
- Apply centralized authentication, policies, guardrails and auditing through Gateway.
- Run on demand without managing a permanent server.

Important distinction

MCP describes how the agent calls a tool. Lambda is one place where the actual tool logic can execute.

6. AgentCore Gateway

What	Gateway is the managed, secure bridge between agents and external tools, APIs, Lambdas and services.
Why	External tools require centralized discovery, authentication, authorization, routing, governance and reuse.
How	Gateway exposes targets as tools, validates incoming credentials, evaluates policies and guardrails, invokes approved targets and sends the result back.

6.1 Gateway Checks

Request arrives -> JWT authentication -> Cedar authorization -> Guardrail content check -> Target invocation

- Authentication: Is the JWT genuine, trusted and unexpired?
- Authorization: Is this user or agent allowed to call this tool with these inputs?
- Guardrail: Is the text or content entering the tool safe?
- Routing: Which Lambda, API or service should receive the approved request?

6.2 Runtime vs Gateway

AgentCore Runtime	AgentCore Gateway
Runs the agent	Connects the agent to remote services
Uses model, prompt and memory	Exposes and routes tools
Chooses tools through the model	Authenticates and authorizes calls
Executes local tools	Invokes approved remote backends
Creates final response	Returns tool results to Runtime

Memory trick

Runtime thinks and responds. Gateway secures and routes. Lambda performs the business task.

7. MCP: Model Context Protocol

What	MCP is a standard protocol for AI applications to discover and invoke external tools.
Why	Without a common protocol, every tool would require separate custom integration and discovery logic.
How	An MCP client connects to an MCP server, lists its tools, sends a tool call with structured arguments and receives the result.

7.1 MCP Components

- MCP Host: the complete AI application, such as the CustomerSupport application.
- MCP Client: the connector inside the application that talks to one MCP server.
- MCP Server: the program or managed endpoint that exposes tools.
- MCP Tool: one callable capability with a name, description and input schema.

7.2 Where MCP Was Used

- Direct MCP connection: Runtime connected directly to the Exa AI MCP server for web-search tools.
- Gateway MCP connection: Runtime connected to AgentCore Gateway, which exposed warranty and refund Lambdas as MCP-compatible tools.
- Local tools did not use MCP because Runtime executed them directly.

7.3 Why No Custom MCP Server Was Needed for Lambda

The company already had Lambda business functions. We supplied the Lambda ARN and tool schema to AgentCore Gateway. Gateway acted as the managed MCP endpoint and exposed the functions as MCP-compatible tools. Therefore, we did not rewrite the Lambda logic inside a custom MCP server.

Existing Lambda -> **Tool schema** -> **Gateway target** -> **MCP-compatible tool** -> **MCP Client can call it**

Precise statement

AgentCore Gateway performs the conversion and exposure. MCP is the standard interface the agent uses to discover and call the converted tool.

8. How One User Request Calls a Tool

8.1 Product Information Request

User asks product price -> Model selects get_product_info -> Runtime executes local function -> Model answers

8.2 Warranty Request

User asks warranty -> Model selects warranty MCP tool -> MCP Client calls Gateway -> Gateway validates and permits -> Warranty Lambda executes -> Result returns

8.3 Refund Request

User asks refund -> Model builds order_id + amount + reason -> Gateway validates JWT -> Policy checks amount -> Guardrail checks reason -> Lambda runs only if allowed

8.4 Multi-Tool Request

For a request such as “Tell me about the USB-C Hub, explain the return policy and check its warranty,” the model may call get_product_info, then get_return_policy, then the remote warranty MCP tool, and combine all results into one response.

9. Viewing MCP Tools in a Personal Project

An MCP server publishes its available tools through the tools/list operation. The response shows tool names, descriptions and input schemas. Local Python tools registered directly with an agent do not appear in the MCP server tool list.

9.1 MCP Inspector

MCP Inspector is the easiest development tool for connecting to an MCP server, viewing available tools and testing calls. Start Inspector, connect the local server command or remote MCP URL, then open Tools and list the published tools.

9.2 What to Verify

- Tool name is clear and unique.
- Description accurately tells the model when to use the tool.
- Input schema contains correct fields and data types.
- Required fields are marked correctly.
- Test calls return structured, useful results.
- Protected servers reject missing or invalid authentication.

> **Important**
>
> tools/list shows tools published by the connected MCP server or Gateway. It does not show unrelated local Python tools.

10. Final Revision Summary

- Cognito manages accounts and verifies login.
- JWT is the temporary signed identity proof used after login.
- Runtime runs the complete coded agent.
- The Foundation Model decides which tool to use.
- Local tools execute directly inside Runtime.
- MCP standardizes remote tool discovery and invocation.
- An MCP client connects the agent to one MCP server.
- An MCP server publishes tools.
- Gateway can act as a managed MCP endpoint for Lambdas and APIs.
- Gateway authenticates, authorizes, checks guardrails and routes requests.
- Lambda performs the actual backend business operation.
- The tool schema explains the backend capability to the model.
- An ARN uniquely identifies an AWS resource.

Customer login -> **JWT** -> **Runtime** -> **Model tool choice** -> **MCP/Gateway** -> **Lambda/API** -> **Final answer**

One-line architecture

Customer logs in through Cognito -> frontend gets JWT -> Runtime runs the agent -> model selects a tool -> MCP carries remote tool calls -> Gateway validates, authorizes and routes -> Lambda performs the business task -> Runtime returns the final answer.